

QR-CP Implementation Review

Repository: `C:\Users\jayao\Desktop\python_projects\DSCP`

Reviewed: September 12, 2026

Scope: core QR pipeline, models, loss, data preparation, configs, and tuning

Verdict

The core implementation correctly follows the intended method for the configured one-step setting. The repository is not fully reliable outside that setting, and the tuning workflow has two serious validity issues.

Findings

HIGH Hyperparameter tuning uses the test split for model selection

The tuner evaluates every trial on `test_data_loader`, derives its selection score from those results, and ranks trials using that score. Consequently, reported test performance after tuning is optimistically biased.

`sbatch_run_tuning/run_qr_cp_tuning.py:169-204` · `:206-220` · `:265-267`

HIGH The tuning grids' `model.window_size` values are silently ignored

Datasets are created once using the base configuration before trial configurations are generated. Both tuning grids nevertheless claim to tune this value, so trials can be labeled with a window size they did not use.

`sbatch_run_tuning/run_qr_cp_tuning.py:232-240` · `:247-252` ·
`configs/qr_cp_configs/qr_rnn_air_tuning_config.yaml:4` · `qr_transformer_air_tuning_config.yaml:5`

HIGH WHEN ENABLED `prediction_step > 1` is not implemented correctly

The models flatten (B, T, H, M) into $(B, T, H * M)$, while the loss combines $(B, H, 1)$ targets with $(B, 1, H * M)$ predictions and an $(M,)$ quantile vector. This fails or computes incorrect Cartesian comparisons. Current-feature concatenation and inference indexing are also one-step-only.

All current QR experiment configurations specify `prediction_step: 1`, so present configured runs avoid this defect.

`dscp/models/qr_rnn.py:59-68` · `dscp/models/qr_transformer.py:63-72` · `dscp/loss.py:15-31` and `:46-61` ·
`dscp/run_qr_cp.py:213-218` and `:498-503`

MEDIUM Saved interval fields contain residual offsets, not final endpoints

`upper_interval` and `lower_interval` store predicted residual quantiles rather than $\hat{f} + q$. Metrics and plots correctly add the base prediction later, so internal reported results are correct, but consumers of `log.pkl` could reasonably misinterpret these fields.

`dscp/run_qr_cp.py:242-252` and `:527-537` · reconstruction in `utils/reporting.py:73-103` and `utils/plotting.py:118-132`

LOW Quantile assumptions are not validated

Quantiles are flattened, deduplicated, and sorted without checking $0 < \tau < 1$, exactly two endpoints per interval, or `alpha_h > alpha_l`. Invalid configurations can therefore reach training or reporting without a useful error.

`utils/utils.py:31-61` · `dscp/loss.py:16-31` and `:47-61`

Method correspondence

- Residuals are defined as $y - \hat{f}$, giving the correct sign when quantile offsets are added back to the base forecast.
- Historical windows precede their targets and are split chronologically.
- Both RNN and Transformer variants use one shared sequence encoder.
- Configured levels are converted into a fixed, sorted set of unique quantiles.
- The lowest quantile is predicted directly; later outputs use softplus increments and cumulative addition.
- The pinball-loss expression uses the same sorted quantile ordering as the model outputs.
- Interval endpoint indices are selected by quantile value rather than configuration order.
- Residual-domain coverage is algebraically equivalent to checking whether the target lies in $[\hat{f} + q_l, \hat{f} + q_h]$.

Key references: `dscp/data.py:33-35, 68-78, 125-155` · `dscp/models/qr_rnn.py:22-68` · `dscp/models/qr_transformer.py:29-72` · `dscp/loss.py:5-63` · `dscp/run_qr_cp.py:213-240, 498-525`

Sign correction in the method description. The displayed recurrence subtracts a nonnegative increment, which would make successive quantiles decrease. The implementation uses addition. That matches the prose (“added”), the stated noncrossing objective, and the required ordering $q_1 \leq \dots \leq q_M$.

Conformal-validity caveat. Neither the supplied description nor the implementation performs a conformal calibration or rank-adjustment step. The implemented procedure is conditional residual quantile regression; by itself, it does not provide the usual finite-sample conformal coverage guarantee.

Verification notes

The review was read-only and the repository worktree was left clean. No tracked automated tests were found. Dynamic model checks could not be executed because PyTorch is unavailable in the active interpreter; the multi-step defect follows directly from the documented tensor shapes and operations.

Prepared from a static audit of repository revision `aa0a8e4` . No implementation files were changed.